PayPal AI Hackathon 2026 · Project plan

Milestone

AI escrow for freelance work: the money is held, the AI checks the work, and the freelancer gets paid the moment it passes.

In one sentence: a client funds a milestone with a PayPal hold, the freelancer delivers, an AI reviewer checks the work against criteria both sides agreed up front, and the platform captures the hold and pays the freelancer out with PayPal Payouts. If something goes wrong, the AI writes the feedback, the refund proposal or the dispute evidence.

What's inside	Page
1. Why this idea, and how it maps to judging	2
2. How the PayPal sandbox works (verified on our account)	3
3. The product: end-to-end workflow	5
4. Milestone lifecycle and the AI reviewer	6
5. Architecture and the PayPal API map	8
6. How each sponsor helps	9
7. Demo script, build plan, risks	10

Prepared Oct 20, 2026. Sandbox results come from live calls on the "hackathon" US sandbox app that day. Items marked Docs come from PayPal's documentation and haven't been run yet.

1. Why this idea

The problem. Freelancers deliver work and then chase payment for weeks, or never get paid. Clients who pay up front sometimes get work that doesn't match the brief. Big marketplaces offer escrow, but with high fees and slow, manual approvals. People working directly with a client, often across borders, have nothing fair in between. Both sides need a neutral third party that holds the money and checks the work.

Who it's for. Independent freelancers (developers, designers, writers) and the small businesses that hire them directly, outside the big marketplaces. The first version targets **software deliverables** (GitHub pull requests), because an AI can check them against tests and code objectively. Written and design work are the next step.

Why it's different. Most "AI + payments" entries will be chatbots that recommend. In Milestone, the AI decides whether money should move, while PayPal's hold-capture-payout rails and fixed rules keep it safe. It's a concrete answer to the hard agentic-commerce question: how do you let an agent move money without trusting it blindly?

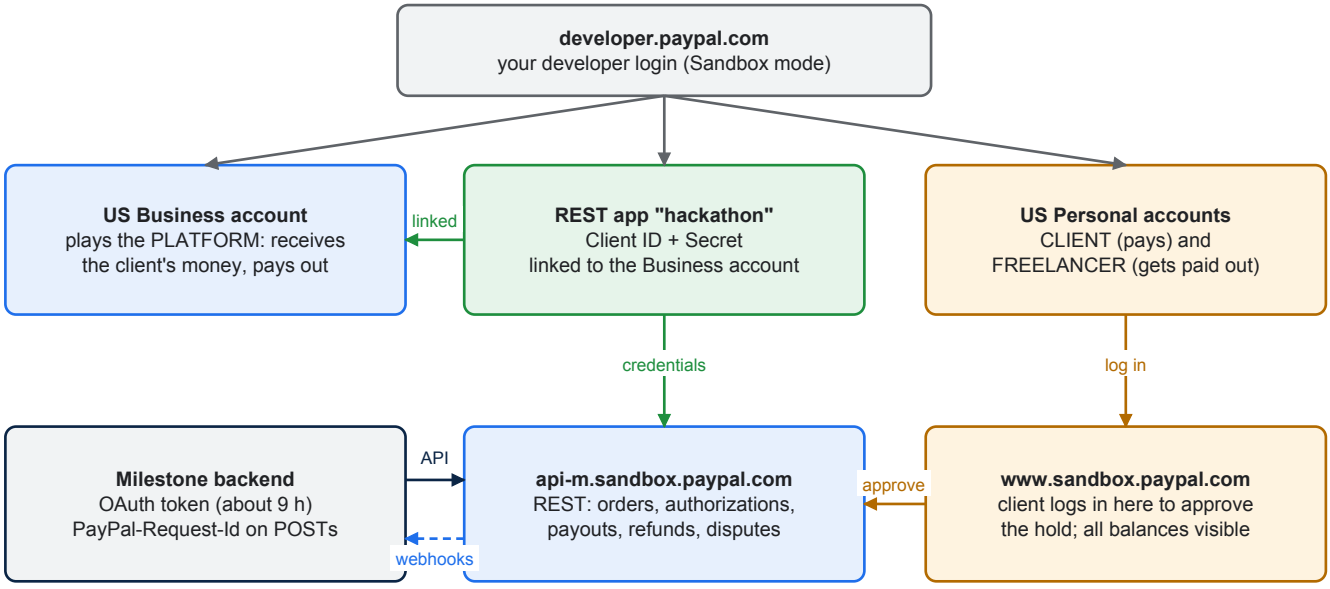
How it maps to the judging criteria

Criterion (equal weight)	How Milestone scores
Technological implementation (also the first tie-breaker)	Real use of PayPal: orders with intent AUTHORIZE, authorize, capture, void, reauthorize, Payouts, refunds, disputes, verified webhooks, idempotency keys. AI is a tool-using agent that runs tests and reads diffs, not one prompt.
Design	A complete product: brief, criteria, funding, timeline, review report, payout receipt. The Bryntum timeline shows money and work on one screen.
Potential impact	A specific audience (independent freelancers and direct clients) and a specific pain (non-payment, disputes over "done"). The demo shows the pain being solved.
Innovation	AI acceptance testing used as the release condition for escrow. Holds plus a fixed decision policy make AI-triggered payments safe.
Presentation	A built-in story: the money is visibly held, then released or blocked, with balances changing on screen.

Prizes it can compete for: Grand Prize; Best Use of Agentic Commerce; Best Use of PayPal + AI; Most Impactful; Most Creative. Plus **one** sponsor prize; the best fit is Bryntum (see section 6). Rule: a project can win at most one Grand or Honorable Mention prize plus one sponsor prize.

2. How the PayPal sandbox works

The sandbox is a copy of PayPal with fake money. Your developer login owns **sandbox accounts** (fake people and businesses) and **REST apps** (API keys). Each app is linked to exactly one business account, and that business account is the merchant behind every API call.



Sandbox money is fake but behaves like production. Sandbox emails are not delivered: check Dashboard > Sandbox > Notifications, and show status in the app itself.

Figure 1. The parts of the sandbox and how Milestone uses them.

Who plays whom in Milestone

Sandbox account	Role in Milestone	To do
US Business (linked to the "hackathon" app)	The platform. Receives the client's captured money and sends Payouts.	Done
US Personal #1	The client. Logs in at sandbox.paypal.com to approve the hold.	Done
US Personal #2	The freelancer. Receives the payout, so the demo can show their balance going up.	**Create this one**

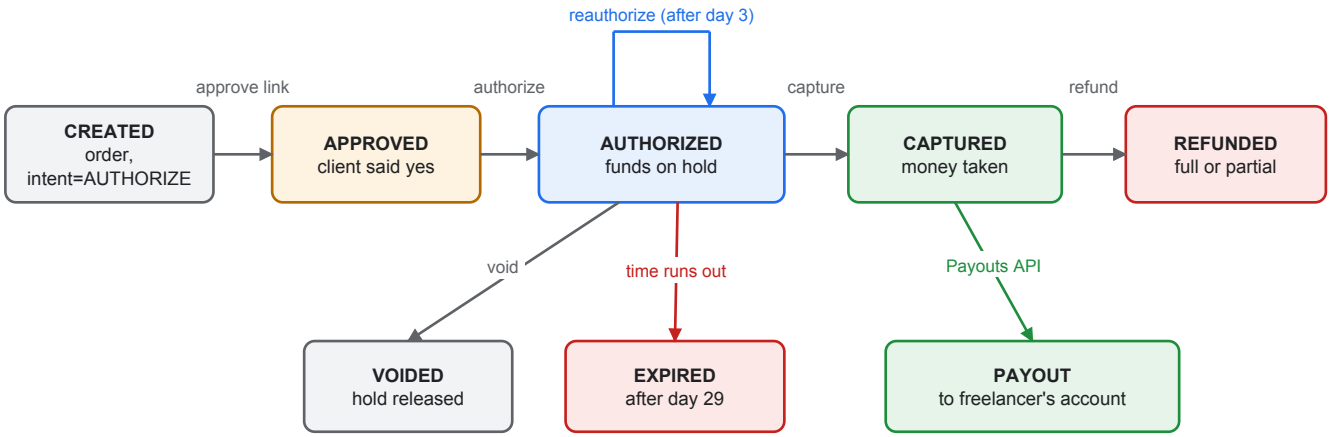
What works on our sandbox app (tested Oct 20)

Capability	Status	Notes
Orders: create, buyer approval link	**Works**	Orders without payment_source start CREATED with approve and authorize links.
Capture before approval	**Works** (fails as expected)	Returns 422 "payer has not yet approved". This is the human consent step.
Payouts	**Works**	Batch accepted with status PENDING (normal; it completes asynchronously).
Card payments + test declines	**Works**	$12 captured with a test Visa. Name CCREJECT-IF gives DECLINED with code 5120.
Vault: save a card	**Works**	Setup token APPROVED. Lets us charge a saved card later.
Vault: save a PayPal wallet	**Blocked**	Limited-release feature. Not needed: holds cover our use case.
Subscriptions, refunds, disputes	**Works** (scopes present)	Refunds need the capture id. Disputes are opened by hand from the buyer account.
Invoicing	**Pending**	Returns 403 while the feature switches on. Milestone doesn't depend on it.

Capability	Status	Notes
Transaction search	**Pending**	Same as invoicing. Use webhooks for real-time state, not reporting.
Negative testing header	**Works**	PayPal-Mock-Response forced INSTRUMENT_DECLINED. Useful for demoing recovery.

The one money pattern Milestone relies on: authorize now, capture later

A normal checkout captures money straight away. Milestone instead creates the order with `intent: AUTHORIZE`. When the client approves, PayPal puts a **hold** on their funds: the money is reserved but not taken. We capture the hold only when the work passes, or void it if the job is cancelled. Nothing is captured until the work is checked, which is exactly how escrow should behave.



Hold = authorization. Honor period 3 days; valid up to 29 days. Refunds need the CAPTURE id, not the order id.

Figure 2. Order and authorization states (PayPal Orders v2 + Payments v2).

Sandbox rules to design around

- **A human has to approve.** No API can approve a wallet payment for the buyer. The client clicks the approve link once per milestone. That's our consent moment, so show it in the demo.
- **Holds expire.** The honor period is 3 days and a hold is valid for up to 29 days. A daily job (refreshHolds) reauthorizes after day 3. Milestones are capped at about 3 weeks.
- **Refunds use the capture id**, not the order id. Store every id PayPal returns.
- **Idempotency.** Send `PayPal-Request-Id` on every POST that moves money, so a retry never pays twice.
- **Webhooks need public HTTPS** and signature verification (verify-webhook-signature). Deploy to Render early so PayPal can reach us.
- **Emails don't arrive.** Show every status change in the app itself. Sandbox emails only appear under Dashboard > Notifications.
- **Tokens last about 9 hours.** Refresh them in code; never hard-code one.

3. The product: end-to-end workflow

Four phases: **Agree** on what "done" means, **Fund** it with a hold, **Deliver + Review** with the AI, and **Release** the money. Each arrow below is either a screen in the app, a PayPal API call or an AI call.

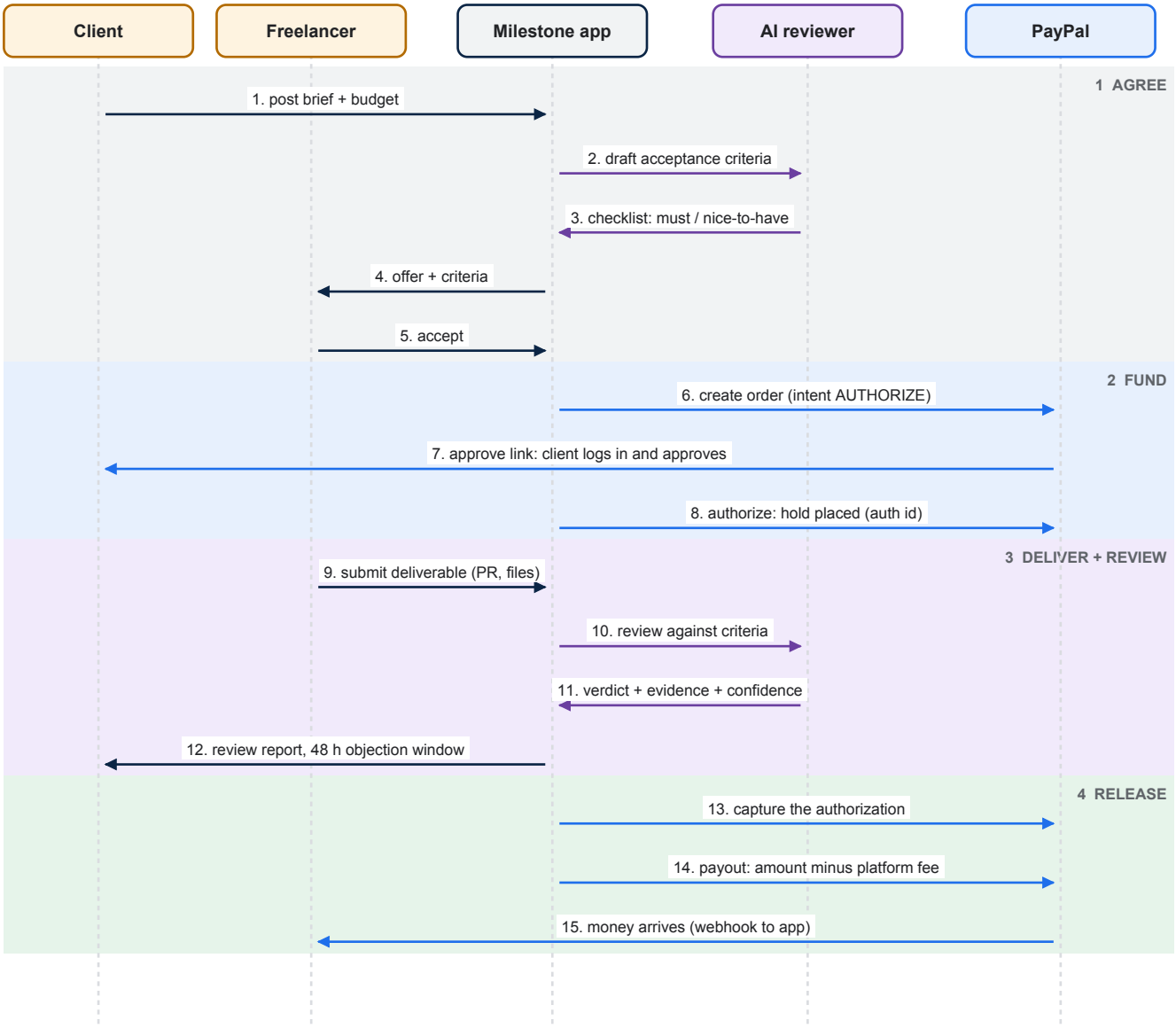


Figure 3. Happy-path sequence. Blue arrows are PayPal calls, purple arrows are AI calls.

> **Why the criteria come first:** the AI turns a vague brief ("build a login page") into a checklist both sides accept before money is held: must items ("passes the CI test suite", "supports email + password") and nice-to-have items. The later review is then objective, which is what makes it fair to let it release money.

4. Milestone lifecycle and the AI reviewer

Every milestone is a small state machine stored in the database. Each change is caused by a person, by the AI's verdict passed through the decision policy, or by a PayPal webhook, and each one is written to the audit log.

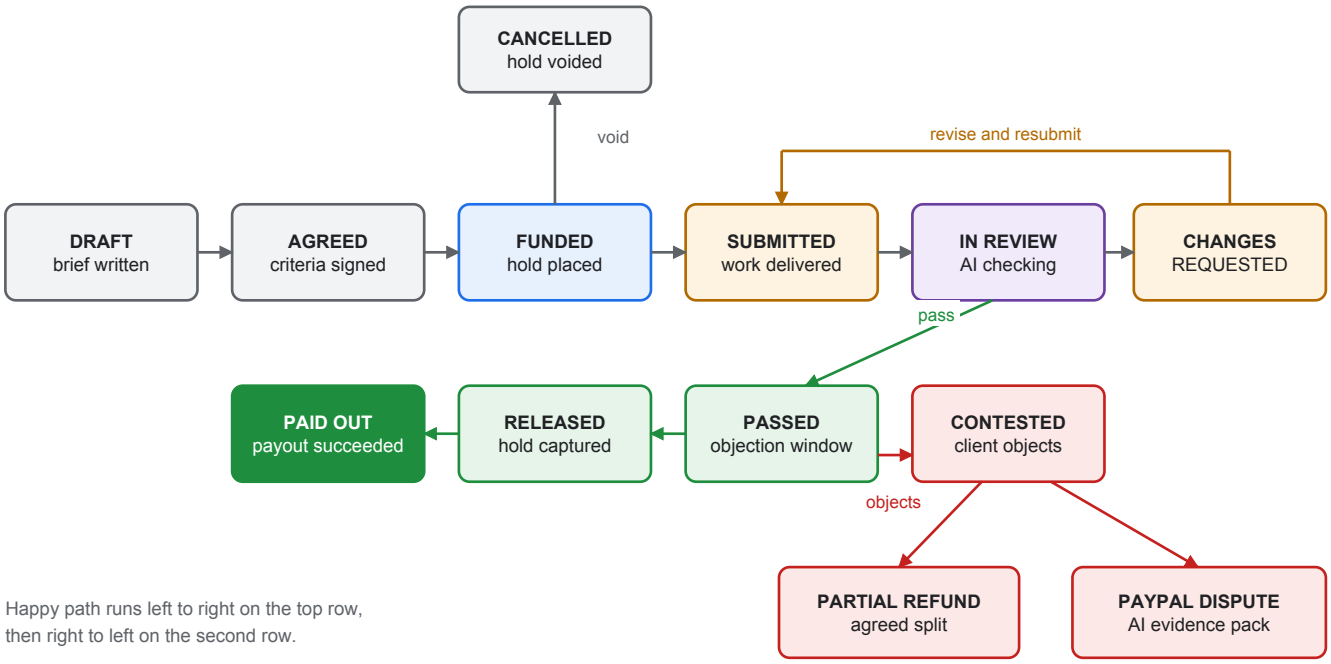


Figure 4. Milestone states. Green = money moving to the freelancer, red = money going back to the client.

Transition	Triggered by	PayPal call
AGREED to FUNDED	Client approves at the PayPal link	create order (AUTHORIZE), then authorize
FUNDED to CANCELLED	Both sides cancel before submission	void authorization
IN REVIEW to PASSED	Decision policy: all musts pass, confidence >= 0.8	none yet
PASSED to RELEASED	Objection window ends (48 h; 2 minutes in the demo)	capture authorization
RELEASED to PAID OUT	Capture completed (webhook)	Payouts (amount minus fee)
CONTESTED to PARTIAL REFUND	Both accept the AI-proposed split	capture partial amount, void rest
CONTESTED to DISPUTE	No agreement	AI drafts the evidence for the dispute

Inside the AI reviewer

The reviewer runs on free OpenRouter models (50 requests a day), so our code first gathers the evidence: the PR diff, changed files and CI results. The model then judges everything in one or two calls, and returns structured output: one verdict per criterion, with quoted evidence and a confidence score. If a model is down or rate-limited, the app falls back to the next one. Then **plain code**, not the model, applies the decision policy. The AI never calls the capture or payout APIs directly.

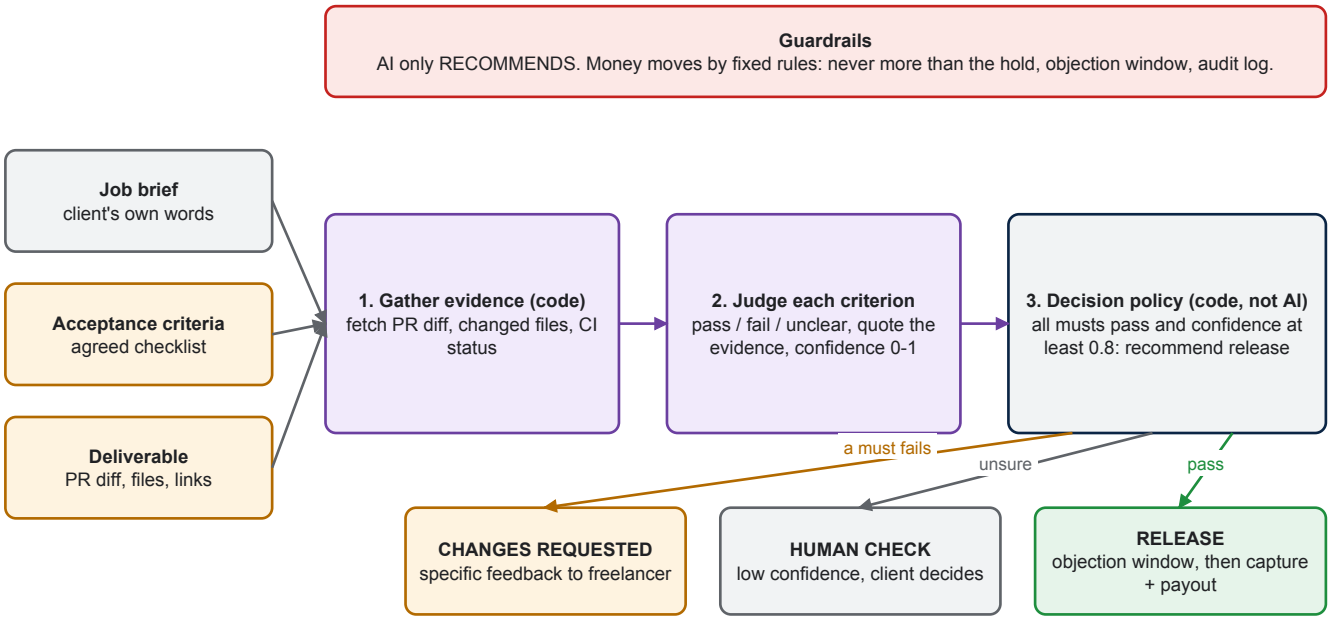


Figure 5. Reviewer pipeline: the AI collects evidence and judges; fixed rules decide whether money moves.

Agent	Input	Output	Model
Criteria writer	Client brief, budget, deadline	Checklist of must and nice-to-have criteria, each testable	AI_MODEL (MiniMax M3.1 Flash)
Reviewer	Criteria + deliverable (PR, files)	Per-criterion verdict, evidence, confidence (JSON)	AI_MODEL, with fallback to Nemotron 3.5, then Gemma 4
Mediator	Objection text + review report	Fair partial-refund proposal with reasoning	AI_MODEL
Dispute writer	Timeline, criteria, verdicts, messages	Evidence summary for a PayPal dispute	AI_MODEL

Free models change without notice. Models are env settings (AI_MODEL, AI_MODEL_FALLBACK, AI_MODEL_FALLBACK_2); check them with scripts/ai-check.sh.

5. Architecture and the PayPal API map

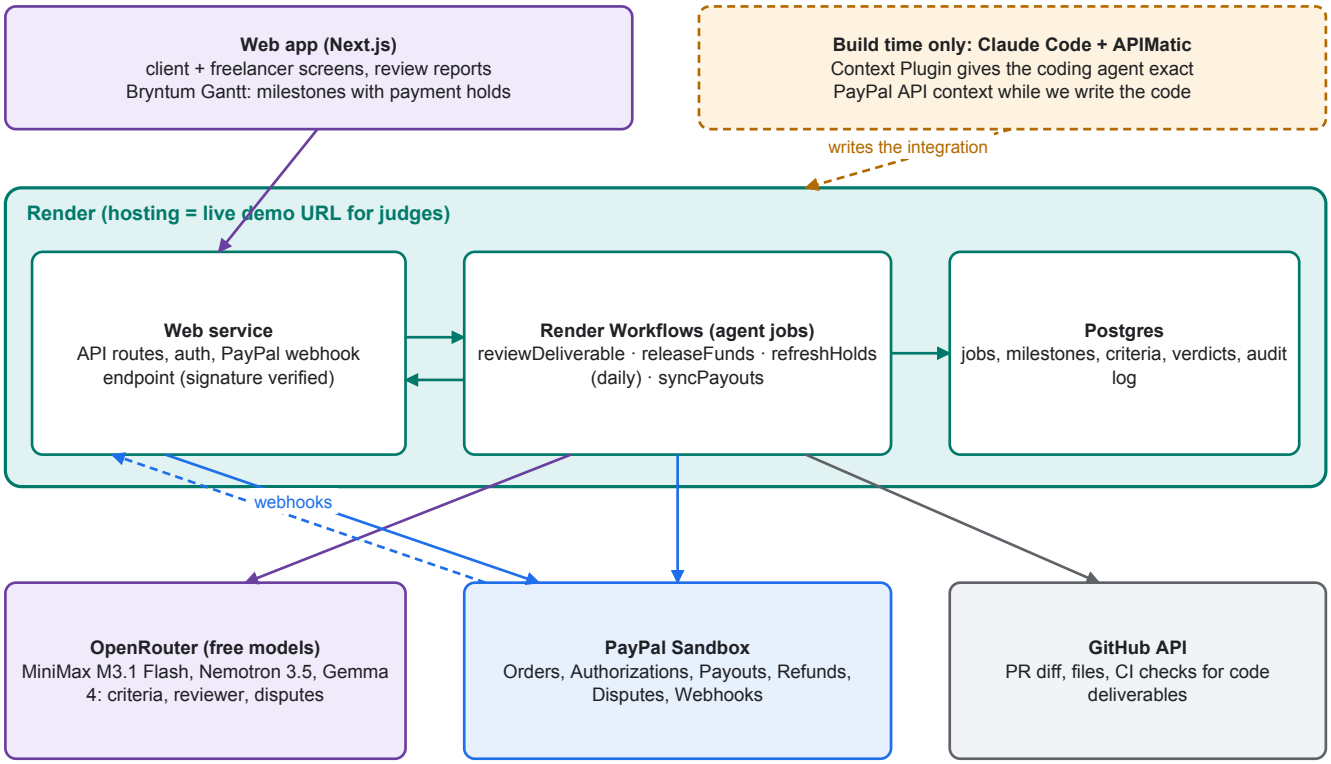


Figure 6. System architecture. Colours show where each sponsor's product sits.

PayPal API map

Step	API call	Endpoint	Tested?
Create hold order	Orders v2 create, intent AUTHORIZE	POST /v2/checkout/orders	**Works**
Client approval	Redirect to approve link	sandbox.paypal.com/checkoutnow	**Works**
Place hold	Authorize order	POST /v2/checkout/orders/{id}/authorize	**Docs**
Release money	Capture authorization	POST /v2/payments/authorizations/{id}/capture	**Docs**
Cancel	Void authorization	POST /v2/payments/authorizations/{id}/void	**Docs**
Keep hold alive	Reauthorize	POST /v2/payments/authorizations/{id}/reauthorize	**Docs**
Pay freelancer	Payouts batch	POST /v1/payments/payouts	**Works**
Refund	Refund capture	POST /v2/payments/captures/{id}/refund	**Docs**
Disputes	List, get, provide evidence, accept	/v1/customer/disputes/...	**Docs**
Webhooks	Register + verify signature	/v1/notifications/webhooks, verify-webhook-signature	**Docs**
Card instead of wallet	Order with payment_source.card	POST /v2/checkout/orders	**Works**

Webhooks to handle: CHECKOUT.ORDER.APPROVED, PAYMENT.AUTHORIZATION.CREATED / VOIDED, PAYMENT.CAPTURE.COMPLETED / REFUNDED, PAYMENT.PAYOUTS-ITEM.SUCCEEDED / FAILED / UNCLAIMED, CUSTOMER.DISPUTE.CREATED / RESOLVED.

Agent Toolkit vs raw REST. PayPal's Agent Toolkit (v1.11, 43 tools) covers orders, capture, refunds, disputes and tracking. Authorize, void, reauthorize, Payouts and webhooks aren't in it, so we write small typed REST functions for those. Because the money calls run in our own code, not as tools the model can call, the AI can't move money on its own.

6. How each sponsor helps

We use three sponsor tools, each because it makes the product better, not just to qualify for a prize. Only one sponsor prize can be won, so **Bryntum is the main target**. APIMatic and Render support the build and the demo.

Sponsor	What it does in Milestone	Why it helps the score	Effort / cost
Bryntum (main sponsor target)	Gantt chart of each project: milestones as tasks, with payment holds and approvals as dependencies. A milestone whose hold is missing or expired is visibly blocked; when the AI passes it, the bar turns green and the next phase unlocks. The agent drags dates when work is late.	Matches Bryntum's own listed ideas almost exactly ("payment tranches as dependencies, a late PayPal payment visibly blocks the next phase" and "an agent watches payment events and moves milestones"). Big boost to Design.	45-day free trial (npm trial package). About 2-3 days of UI work. Prize: 3 x $1,000.
APIMatic	Context Plugin installed in Claude Code while building, so the coding agent writes PayPal calls (authorize, reauthorize, payouts, webhook verify) from accurate, current API context.	Fewer integration bugs and faster build. Mention it in the README and video. Counts for its prize and a free month of their plan.	Free; about 15 minutes to set up. Prize: 3 x $1,000 + 6 months Business plan.
Render	Hosts the web service and Postgres (a live URL judges can click). Render Workflows run the agent jobs: reviewDeliverable, releaseFunds after the objection window, the daily refreshHolds, and payout sync.	Judges must be able to run the project, and a hosted URL is the easiest way. Workflows give the agent reliable background execution with retries. Also gives PayPal a public HTTPS URL for webhooks.	$50 credits from the hackathon. About 1 day. Prize: Render credits only.

Sponsors we're skipping, and why

Sponsor	Decision
AG Grid	Optional stretch goal: an admin "escrow ledger" dashboard (holds by age, payouts, disputes). Only if time allows; done half-way it would hurt Design.
Channel3	Product-catalog data. Doesn't fit freelance services.
Kernel	Browser agents. Could check live websites as deliverables later; too risky for the demo.
Elastic	Search over past reviews or disputes. Not needed at this scale.
Astropods, Zapier, Postman	No prize or no clear role. Postman is still handy for testing PayPal calls by hand.

7. Demo script, build plan, risks

3-minute demo storyboard

Time	On screen	The point
0:00-0:20	A freelancer's "unpaid invoice" story; client and freelancer side by side	The problem and who it's for
0:20-0:45	Client types a brief; AI writes the acceptance checklist; freelancer accepts	AI makes "done" objective
0:45-1:10	Client approves on the PayPal sandbox page; the Bryntum bar shows "Funded: $300 held"	Real PayPal hold, human consent
1:10-1:50	Freelancer submits a PR; the AI reviewer runs and the per-criterion report fills in live	AI as reviewer, with evidence
1:50-2:20	Objection window ends; capture; payout; freelancer's sandbox balance goes up	Money actually moves
2:20-2:45	Second milestone: a bad PR is rejected with specific feedback, and the money stays held	Guardrails work
2:45-3:00	Architecture slide: PayPal APIs, OpenRouter AI, Bryntum, Render; closing line	Technical depth + vision

Build plan (Oct 21 to Nov 12)

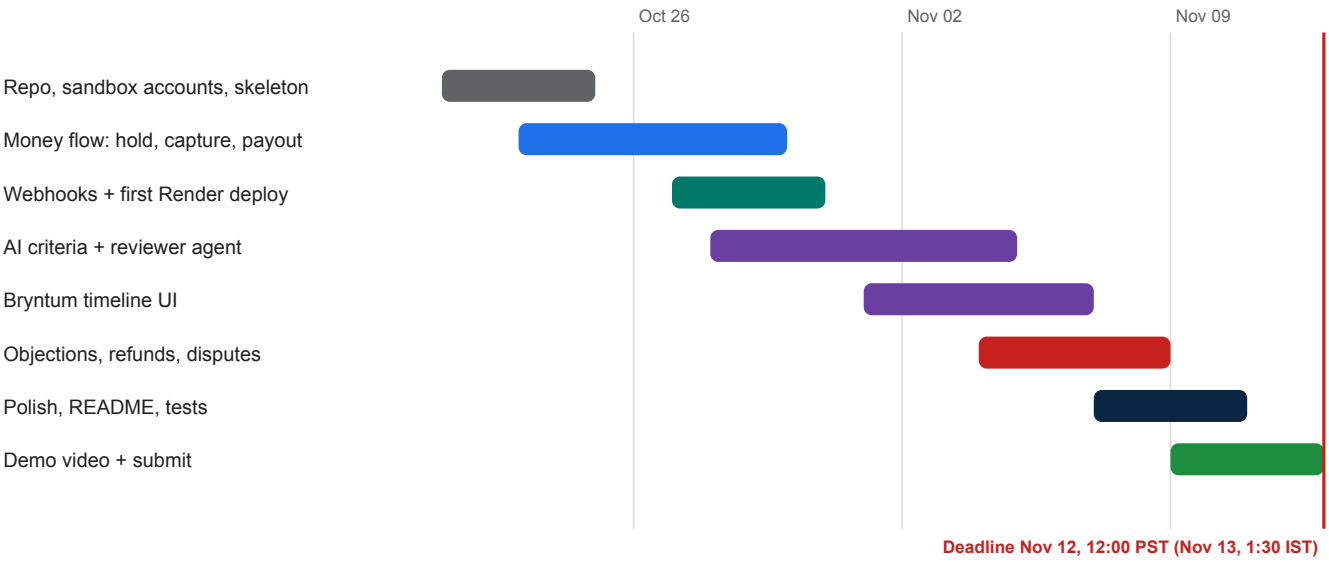


Figure 7. Suggested schedule. Get the money flow working first; it's the riskiest part and the core of the demo.

Risks and how we handle them

Risk	Mitigation
The AI wrongly passes or fails work	Fixed decision policy, a confidence threshold, a human-check outcome and a client objection window. The AI recommends; rules decide.
Holds expire after 29 days	Milestones capped at about 3 weeks; daily reauthorize job; void and re-fund if needed.
Real-world compliance (holding money for others)	In production this would run on PayPal's marketplace (Commerce Platform) onboarding. Say so in the pitch as the roadmap.
Payouts finish asynchronously	Track the payout item status by webhook; show "Paying out..." until it succeeds.
Invoicing still 403	Not needed for Milestone. If it switches on, add invoice receipts as a bonus.
Judges can't run it	Hosted on Render, plus a README with sandbox setup and test logins for the client and freelancer accounts.
Scope creep	Code (GitHub PR) deliverables only for the hackathon; design and docs review are roadmap.